Documento Final de Arquitectura: Re-arquitectura del Módulo MVP de PaladinesFC

Sistema PaladinesFC — Módulos Jugadores, Jugador–Tutor, Tutores y Profesores

Entrega final (Semana 15)

Giany Andrés Salazar Bustamante

Mariana Aguilar Alcaraz

Carlos Daniel Palacios Murillo

Juan Camilo Moreno Pérez

Facultad de Ingeniería, Universidad Católica Luis Amigó

Arquitectura de Sistemas

Fernando Higuita Betancur

5 de noviembre de 2026

Repositorio: https://github.com/MarianaA27/ClubDeFutbol__

Documento Final de Arquitectura: Re-arquitectura del Módulo MVP de PaladinesFC

Este documento cierra el trabajo del curso sobre PaladinesFC, la aplicación de escritorio que gestiona un club de fútbol formativo. En la Semana 3 se diagnosticaron cinco problemas arquitectónicos; en la Semana 8 se priorizaron los atributos de calidad y sus escenarios; en la Semana 13 se diseñó la arquitectura objetivo y se registraron las decisiones de refactorización (Salazar Bustamante et al., 2026c). Aquí se presenta cómo quedó realmente el sistema: su vista lógica, los diagramas C4 de contexto, contenedores y componentes, el diagrama de despliegue, el flujo principal y la sección de refactorización realizada, con los resultados medidos antes y después del cambio.

Además, se atienden los dos comentarios de la retroalimentación del Entregable 3: las transacciones y la preservación del historial se validaron contra SQL Server real, y cada escenario de calidad tiene una medición antes/después sobre la misma operación, los mismos datos y el mismo servidor. El código, las pruebas, los scripts de medición y la evidencia de funcionamiento están en el repositorio del proyecto (Salazar Bustamante et al., 2026d).

Descripción del Sistema y del Módulo MVP

PaladinesFC se desarrolló el semestre anterior, en la asignatura de Bases de Datos, con Python 3, CustomTkinter y SQL Server. Reemplazó el manejo en papel del club: jugadores y sus acudientes, cuerpo técnico, plantillas y alineaciones, categorías, canchas, sesiones de entrenamiento con asistencia, conceptos de pago, caja, morosidad y un tablero de indicadores. La usa una sola persona, la administradora del club, en un equipo con Windows.

El módulo MVP de la re-arquitectura reúne **Jugadores, Jugador–Tutor, Tutores y Profesores**, con 18 operaciones (Salazar Bustamante et al., 2026a). Se eligió porque todos los demás módulos dependen de esas entidades y porque concentra los cinco problemas

estructurales. Los módulos restantes siguen funcionando sobre el código original, aislado y depurado en `src/paladinesfc/legado/`, y se pueden migrar después con el mismo patrón.

El criterio de éxito fue que la usuaria no percibiera cambios en pantallas, campos ni flujo, salvo las correcciones deliberadas: "Eliminar" pasó a ser "Dar de baja" y los desplegables muestran el documento junto al nombre para distinguir homónimos. Como novedad, el MVP también se ofrece como API REST sobre los mismos servicios.

Vista Lógica Final del Sistema

El estilo es un **monolito modular en capas con inversión de dependencias**, una versión simplificada de la arquitectura limpia (Martin, 2017). Se mantuvo una sola aplicación y una sola base de datos, porque el problema no era de despliegue ni de escala sino de responsabilidades mezcladas. La Tabla 1 resume las capas y la Figura 1 su estructura de paquetes.

Tabla 1

Capas del sistema y su responsabilidad

Capa	Paquete	Responsabilidad	Puede depender de
Presentación	`presentacion/` (`app.py`, `api/`, `contenedor.py`)	Interfaz de escritorio y API REST; solo llaman a servicios. La raíz de composición crea los servicios	Aplicación (y la infraestructura solo en la raíz de composición)
Aplicación	`aplicacion/` (`*_service.py`, `puertos.py`)	Casos de uso del MVP; define los puertos (contratos) de repositorios y unidad de trabajo	Dominio y sus propios puertos
Dominio	`dominio/`	Entidades `Jugador`, `Tutor`, `Profesor`, agregado `TutoresDelJugador`, validaciones y errores	Nada
Infraestructura	`infraestructura/`	Unidad de trabajo, repositorios SQL, mapeo por nombre, configuración y bitácora; implementa los puertos	Dominio y aplicación (puertos)
Legado	`legado/`	Acceso a datos de los módulos fuera del MVP	Configuración

Nota. Las dependencias apuntan hacia el dominio: el dominio no importa pyodbc ni CustomTkinter, y los servicios no dependen de la infraestructura sino de sus puertos.

Figura 1

Estructura de paquetes del sistema re-arquitecturado

```
src/
├── paladinesfc/
│   ├── presentacion/     app.py (CustomTkinter), api/ (FastAPI), contenedor.py
│   ├── aplicacion/       puertos.py, jugador_service.py, vinculacion_service.py,
│   │                     tutor_service.py, profesor_service.py
│   ├── dominio/          jugador.py, tutor.py, profesor.py, vinculo.py,
│   │                     validaciones.py, errores.py
│   ├── infraestructura/  config.py, conexion.py, unidad_trabajo.py, mapeo.py,
│   │                     bitacora.py, repositorios/
│   └── legado/           crud.py, db.py (módulos fuera del MVP)
└── sql/
    ├── ClubFutbolDB.sql  esquema original
    └── migraciones/      001 a 004, versionadas e idempotentes
```

Nota. Adaptado de `docs/arquitectura.md`, §3.2, del repositorio del proyecto.

Las reglas de dependencia se verifican en el código: la presentación conoce la infraestructura solo en `presentacion/contenedor.py` (raíz de composición) y en `presentacion/api/errores.py` (registro en la bitácora y reconocimiento de los errores de la base de datos); ninguna pantalla del MVP ejecuta SQL ni lee datos por posición.

Relación con el patrón MVC

La rúbrica pide aplicar patrones básicos como MVC cuando aplique. La separación de PaladinesFC es una variante en capas de esa misma idea. El **modelo** es el dominio con los servicios de aplicación, que contienen todas las reglas. Las **vistas** son las ventanas de CustomTkinter y, en la API, los esquemas de respuesta en JSON. Los **controladores** son los manejadores de eventos de cada ventana y las rutas de FastAPI: reciben la petición, llaman al servicio y devuelven el resultado o el error. Como modelo, vistas y controladores están separados, la API se agregó como segundo adaptador sin tocar las reglas de negocio.

Diagramas C4 del Sistema Implementado

Los diagramas siguen el modelo C4 (Brown, s.f.) y corresponden a la versión 2, que muestra lo que realmente se implementó. Las fuentes editables en PlantUML están en

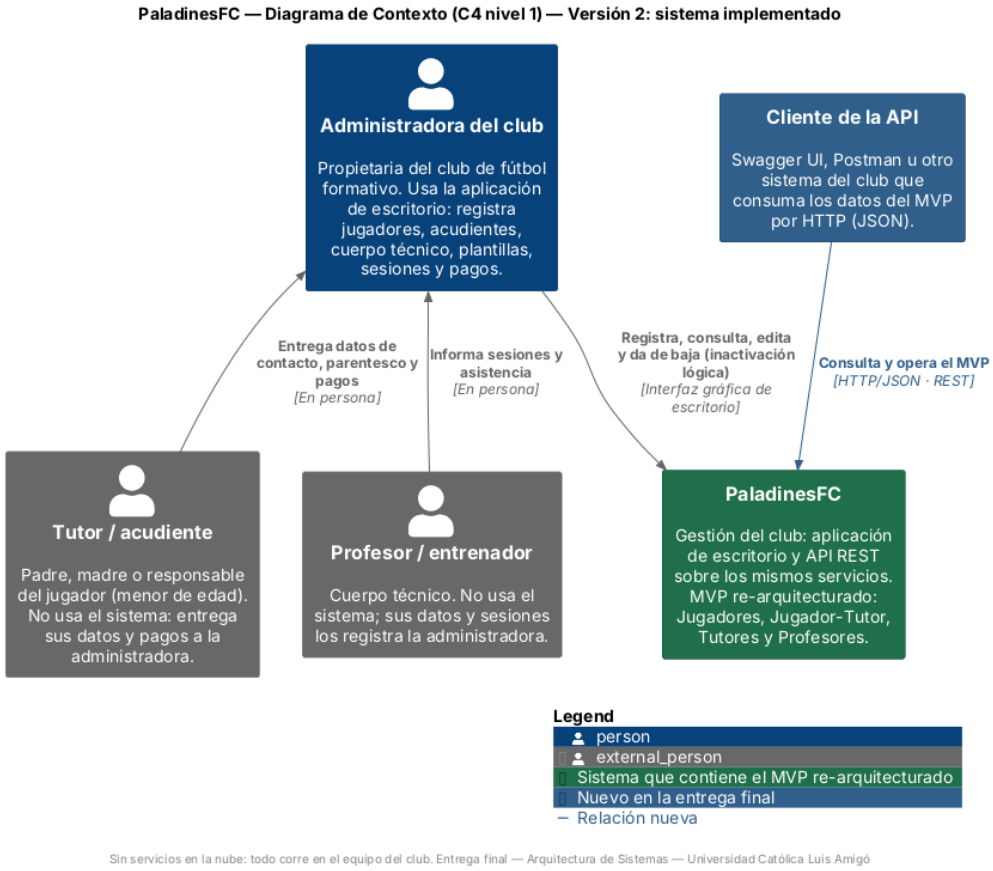
`docs/diagramas/fuentes/` y las imágenes en `docs/diagramas/exportados/`. Frente a la versión 1 del Entregable 3, la versión 2 agrega la API REST, las migraciones 001 a 004 y la integración continua.

Diagrama de Contexto

La Figura 2 muestra a la administradora del club como única usuaria de la aplicación de escritorio y a un cliente de la API (Swagger UI, Postman u otro sistema). PaladinesFC guarda sus datos en SQL Server en el mismo equipo.

Figura 2

Diagrama de contexto (C4 nivel 1), versión 2



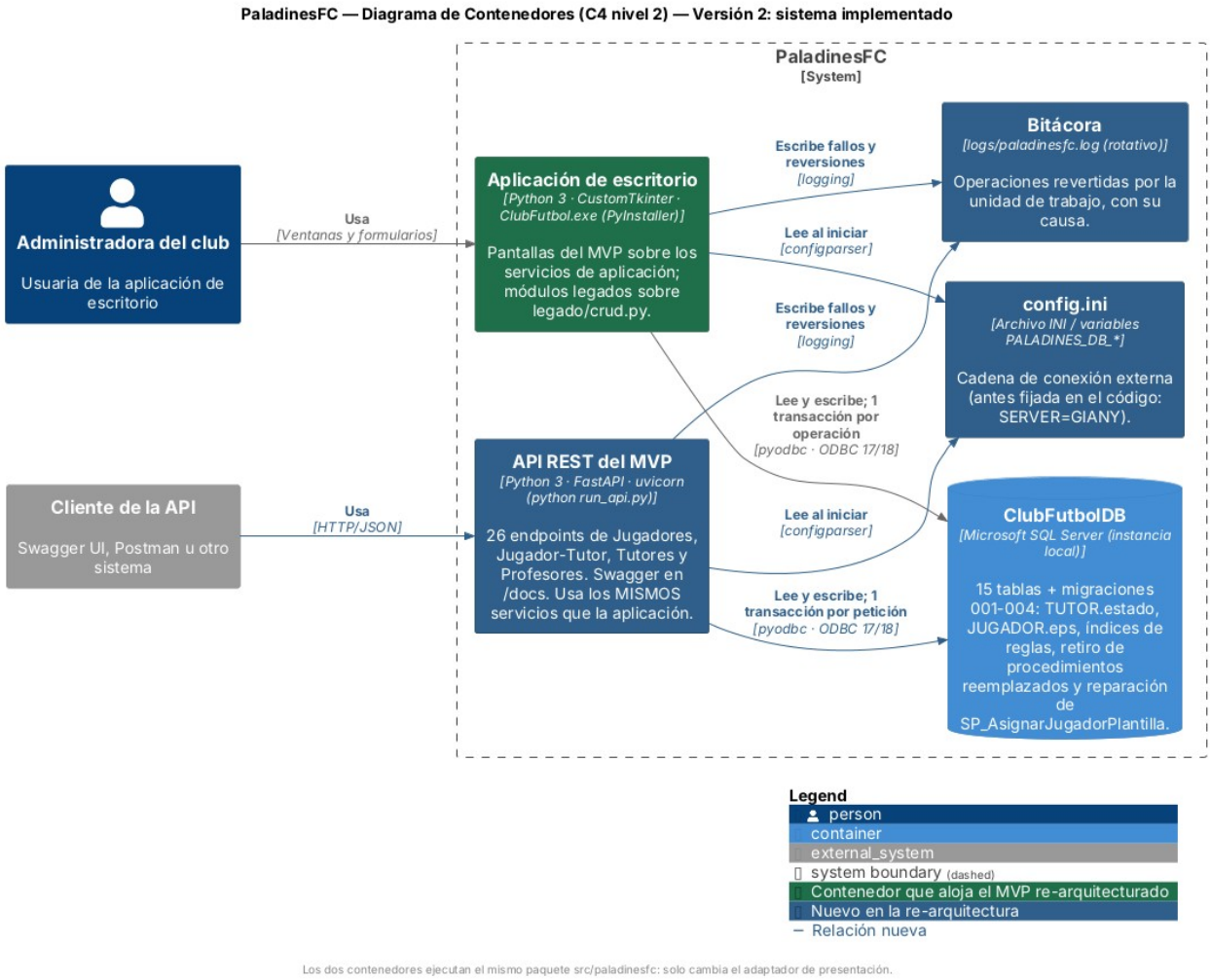
Nota. Elaboración propia. Fuente: `docs/diagramas/fuentes/c4-1-contexto.puml`.

Diagrama de Contenedores

La Figura 3 muestra los dos contenedores ejecutables, la aplicación de escritorio y la API REST, que ejecutan el mismo paquete `src/paladinesfc` y solo cambian el adaptador de presentación. Ambos leen la conexión de `config.ini`, escriben los fallos en la bitácora y usan la base `ClubFutbolDB` con una transacción por operación.

Figura 3

Diagrama de contenedores (C4 nivel 2), versión 2



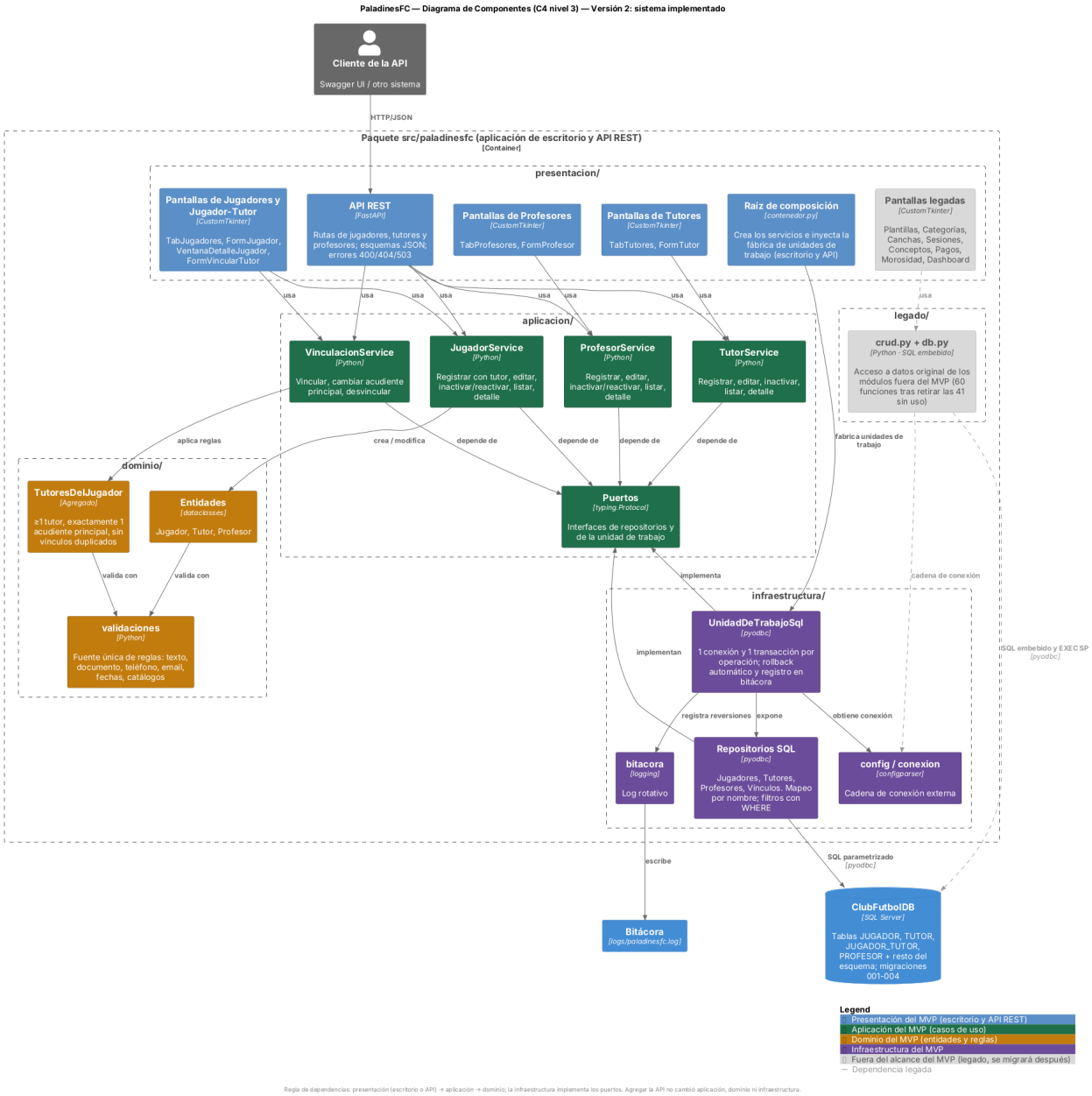
Nota. Elaboración propia. En verde, el contenedor que aloja el MVP re-arquitecturado; en azul oscuro, lo nuevo en la re-arquitectura.

Diagrama de Componentes del Módulo MVP

La Figura 4 descompone el MVP en sus componentes: pantallas y rutas de la API en la presentación; los cuatro servicios y los puertos en la aplicación; entidades, agregado y validaciones en el dominio; unidad de trabajo, repositorios, mapeo, configuración y bitácora en la infraestructura. La Tabla 2 resume cada componente con el código original que reemplaza.

Figura 4

Diagrama de componentes del módulo MVP (C4 nivel 3), versión 2



Nota. Elaboración propia. Fuente: `docs/diagramas/fuentes/c4-3-componentes.puml`.

Tabla 2

Componentes del MVP y código original que reemplazan

Componente	Capa	Responsabilidad	Reemplaza
Pantallas del MVP y rutas de la API	Presentación	Interacción con la usuaria y con clientes HTTP; solo llaman a servicios	`TabJugadores`, `FormJugador`, `TabTutores`, `TabProfesores` y demás de `App.py`
Raíz de composición	Presentación	Crea los servicios y les inyecta la fábrica de unidades de trabajo	No existía
`JugadorService`, `VinculacionService`, `TutorService`, `ProfesorService`	Aplicación	Casos de uso; alta de jugador con su vínculo en una transacción; cambio de principal; bajas lógicas	Funciones del MVP en `crud.py`
Puertos	Aplicación	Contratos de repositorios y unidad de trabajo	No existía
Entidades y agregado `TutoresDelJugador`	Dominio	Reglas de cada entidad; al menos un tutor y exactamente un principal	Tuplas posicionales y validaciones dispersas
`validaciones`	Dominio	Fuente única de formatos y catálogos	`_validar_*`, listas de `App.py`, `CHECK` y SP
`UnidadDeTrabajoSql`	Infraestructura	Una conexión y una transacción por operación; reversión y bitácora	`ejecutar_query` con `commit()` por sentencia
Repositorios SQL y `mapeo`	Infraestructura	SQL parametrizado, filtros con `WHERE` y mapeo por nombre de columna	Consultas de `crud.py` y vistas `VW_Detalle*`
`config`, `conexion`, `bitacora`	Infraestructura	Conexión externa (`config.ini` o variables `PALADINES_DB_*`) y registro de fallos	`SERVER=GIANY` fijo en `db.py`
Migraciones 001 a 004	Base de datos	Cambios de esquema versionados e idempotentes	Alteraciones sueltas al final del script

Nota. Versión resumida de la tabla de componentes de `docs/arquitectura.md`, §4.

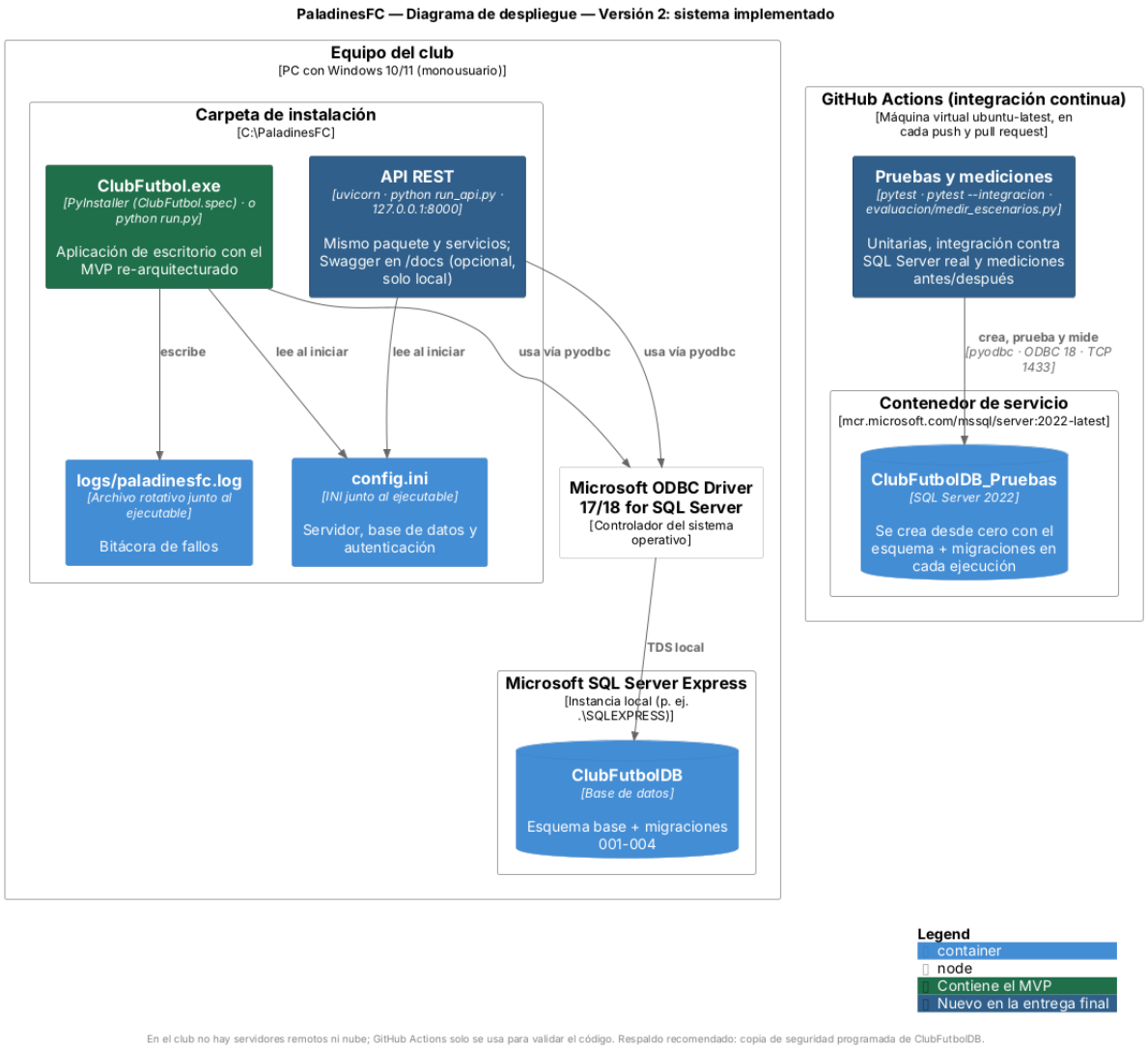
Diagrama de Despliegue

El sistema se ejecuta en local (Figura 5). En el equipo del club, la carpeta de instalación contiene `ClubFutbol.exe`, generado con PyInstaller, junto con `config.ini` y la carpeta `logs/`. La API REST es opcional y se inicia con `python run_api.py` en `127.0.0.1:8000`. Ambos se conectan con el controlador ODBC 17 o 18 a la instancia local de SQL Server Express, donde está `ClubFutbolDB` con el esquema base y las migraciones 001 a 004.

En la nube solo se usa GitHub Actions para validar el código: en cada push y pull request levanta un contenedor con SQL Server 2022, crea desde cero `ClubFutbolDB_Pruebas` con el esquema y las migraciones, ejecuta las pruebas unitarias y de integración y repite las mediciones antes/después.

Figura 5

Diagrama de despliegue, versión 2



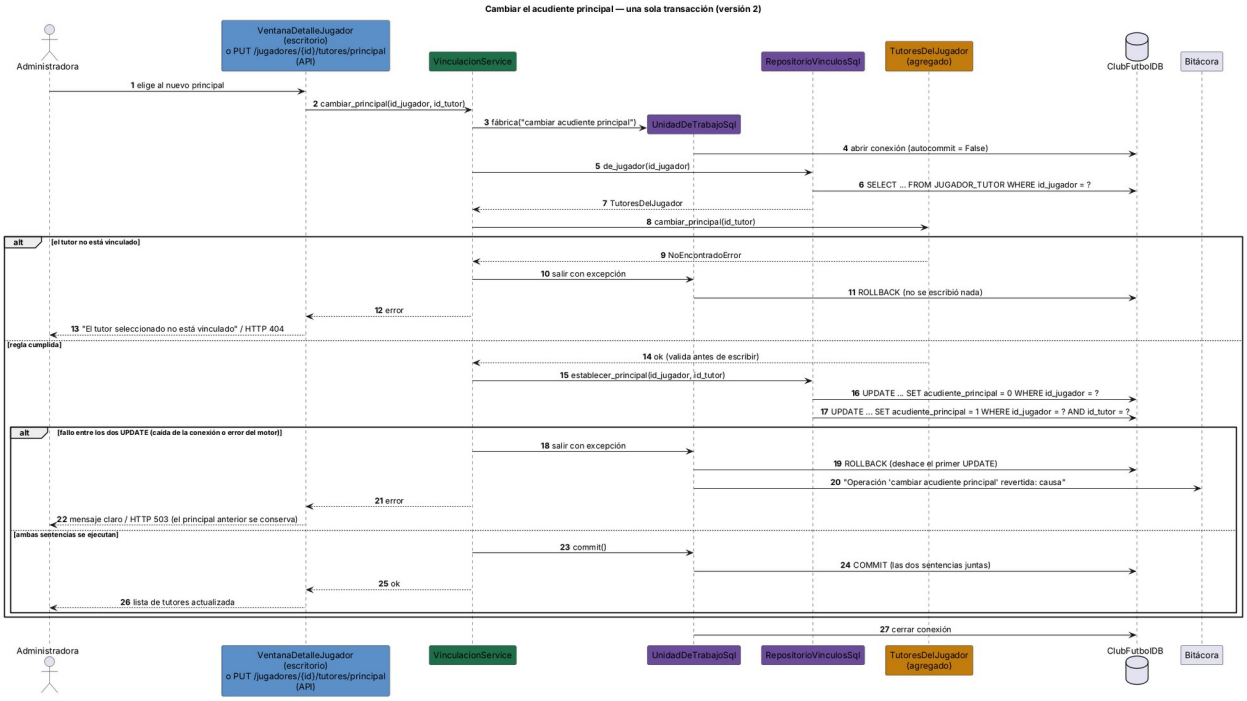
Nota. Elaboración propia. En el club no hay servidores remotos; la nube solo se usa para la integración continua.

Diagrama de Secuencia del Flujo Principal

El flujo elegido es cambiar el acudiente principal de un jugador, porque es el que motivó el escenario de confiabilidad E2: son dos escrituras que antes iban en transacciones separadas. La Figura 6 muestra que la ventana de detalle, o la ruta `PUT /jugadores/{id}/tutores/principal`, llama a `VinculacionService`. El servicio abre una unidad de trabajo, carga el agregado `TutoresDelJugador`, que valida la regla, y guarda los dos cambios. Si algo falla, la unidad de trabajo revierte todo, registra el fallo en la bitácora y la usuaria recibe un mensaje claro.

Figura 6

Secuencia de cambiar el acudiente principal en una sola transacción



Nota. Elaboración propia. Fuente: `docs/diagramas/fuentes/secuencia-cambiar-principal.puml`.

Refactorización Realizada

Problemas de la Versión Inicial

La Tabla 3 recoge los cinco problemas identificados en la Semana 3 con su evidencia en el código original.

Tabla 3

Problemas arquitectónicos de la versión inicial

Id	Problema	Evidencia en el código original
P1	Contrato posicional entre capas	La interfaz leía `j[1]`, `r[6]`…: 98 lecturas por posición en las pantallas del MVP. Los desplegables comparaban el texto visible y en algunos detalles las columnas ya se mostraban corridas
P2	Ausencia de atomicidad	`ejecutar_query()` hacía `commit()` por sentencia; cambiar el acudiente principal eran dos `UPDATE` sueltos
P3	Borrado físico en cascada	`SP_EliminarProfesor` y `SP_EliminarJugador` destruían sesiones, asistencias, plantillas y pagos
P4	Reglas duplicadas sin fuente única	El estado del jugador se validaba en 4 lugares; `SP_ActualizarProfesor` nunca se usaba; nadie garantizaba un único acudiente principal
P5	Capas violadas y consultas ineficientes	`DELETE` escrito en la presentación; búsquedas y ediciones que traían la tabla completa; funciones sin uso y duplicadas en `crud.py`

Cambios Aplicados en el Diseño y el Código

Cada problema se atendió con una decisión registrada como ADR (Nygard, 2011). La Tabla 4 muestra la decisión aplicada y cómo quedó el código.

Tabla 4

Decisiones de refactorización aplicadas

Problema	Decisión	Cómo quedó
P1	ADR-0001: entidades de dominio y mapeo por nombre	Entidades con `desde_bd()` y mapeo por nombre de columna; desplegables resueltos por id. Cero lecturas por posición en las pantallas del MVP
P2	ADR-0002: unidad de trabajo transaccional	`UnidadDeTrabajoSql`: una transacción por operación, reversión y bitácora; conexión configurable fuera del código

Problema	Decisión	Cómo quedó
P3	ADR-0003: inactivación lógica	"Dar de baja" cambia el estado a `inactivo`; la migración 001 agrega `TUTOR.estado` y los índices de reglas y retira los procedimientos de borrado; la 003 retira los de inserción y actualización y las vistas VW_Detalle* que el MVP reemplazó
P4	ADR-0004: fuente única de reglas	Reglas en `dominio/validaciones.py` y en el agregado `TutoresDelJugador`; índices únicos como defensa en la base
P5	ADR-0005: repositorios con filtrado en SQL	Repositorios con `WHERE`; ninguna pantalla ejecuta SQL; se retiraron 41 definiciones de `legado/crud.py`: 39 funciones sin uso y las 2 duplicadas

Servicios y Endpoints REST

Los casos de uso del MVP quedaron en cuatro servicios de aplicación, que son el único punto de entrada tanto para el escritorio como para la API. La API REST, construida con FastAPI, expone 26 endpoints del MVP y una ruta de salud, con documentación Swagger generada en `/docs`. Las reglas son las mismas que en el escritorio: una transacción por operación, baja lógica y selección de tutores por id. La Tabla 5 resume las rutas por recurso; la tabla completa, con ejemplos de entrada y respuesta en JSON, está en el README del repositorio.

Tabla 5

Endpoints REST del módulo MVP

Método	Ruta	Descripción
GET	`/salud`	Comprueba que la API responde
GET · POST	`/jugadores`	Lista o busca por documento; registra un jugador con su acudiente principal en una transacción
GET · PUT	`/jugadores/{id_jugador}`	Consulta o edita un jugador (incluida la EPS)
GET	`/jugadores/{id_jugador}/detalle`	Jugador, sus tutores y sus plantillas activas
POST	`/jugadores/{id_jugador}/baja` · `/reactivacion`	Baja lógica y reactivación
GET · POST	`/jugadores/{id_jugador}/tutores`	Tutores del jugador; vincula otro tutor
PUT	`/jugadores/{id_jugador}/tutores/principal`	Cambia el acudiente principal (dos escrituras, una transacción)
DELETE	`/jugadores/{id_jugador}/tutores/{id_jugador_tutor}`	Desvincula un tutor que no sea el único ni el principal

Método	Ruta	Descripción
GET · POST	`/tutores` · `/tutores/activos`	Lista, busca y registra tutores; activos para listas de selección
GET · PUT	`/tutores/{id_tutor}`	Consulta o edita un tutor
GET	`/tutores/{id_tutor}/jugadores`	Jugadores a cargo del tutor
POST	`/tutores/{id_tutor}/baja`	Baja de un tutor sin jugadores activos a cargo
GET · POST	`/profesores` · `/profesores/activos`	Lista, busca y registra profesores; activos para desplegables
GET · PUT	`/profesores/{id_profesor}`	Consulta o edita un profesor
GET	`/profesores/{id_profesor}/plantillas`	Plantillas a cargo del profesor
POST	`/profesores/{id_profesor}/baja` · `/reactivacion`	Baja lógica (conserva sesiones, asistencias y plantillas) y reactivación

Nota. Errores: `400` cuando una regla de negocio no se cumple, `404` si el registro no existe, `422` si el JSON no es válido y `503` si la base de datos no responde; el cuerpo trae `{"detalle": "mensaje"}` con el mismo texto que ve la usuaria.

La Figura 7 muestra un ejemplo de petición y respuesta del flujo principal.

Figura 7

Ejemplos de entrada y respuesta en JSON

```
POST /jugadores
{"nombres": "Samuel", "apellidos": "Rojas", "documento": "1020304050",
 "fecha_nacimiento": "2015-05-10", "fecha_ingreso": "2024-02-01",
 "talla_uniforme": "S", "numero_preferido": 9, "eps": "Sura",
 "id_tutor": 1, "parentesco": "Padre"}
→ 201 {"id": 3}

PUT /jugadores/3/tutores/principal      {"id_tutor": 2}
→ 200 [ ... vínculos, con "acudiente_principal": true solo en el tutor 2 ]

DELETE /jugadores/3/tutores/5   (vínculo del acudiente principal)
→ 400 {"detalle": "No se puede desvincular al acudiente principal.
        Primero marque a otro tutor como principal."}
```

Nota. Tomado de la tabla de endpoints del README del repositorio.

Patrones Aplicados

La Tabla 6 relaciona los patrones del curso con el lugar donde se aplican y el problema que resuelven (Fowler, 2002; Evans, 2003).

Tabla 6

Patrones aplicados en la re-arquitectura

Patrón	Dónde se aplica	Qué resuelve
Arquitectura en capas	`presentacion`, `aplicacion`, `dominio`, `infraestructura`	Separa responsabilidades y fija el sentido de las dependencias (P5)
Inversión de dependencias (puertos)	`aplicacion/puertos.py`, implementados en `infraestructura/`	Los servicios no dependen de pyodbc y se prueban en memoria
Unit of Work	`infraestructura/unidad_trabajo.py`	Una transacción por operación de negocio (P2)
Repository	`infraestructura/repositorios/`	Concentra el SQL del MVP y filtra en el motor (P5)
Data Mapper	`infraestructura/mapeo.py` y `desde_bd()` de las entidades	Filas convertidas en objetos por nombre de columna (P1)
Aggregate	`TutoresDelJugador` en `dominio/vinculo.py`	Al menos un tutor y exactamente un principal (P4)
Service Layer	`aplicacion/*_service.py`	Un solo punto de entrada por caso de uso para escritorio y API
Inyección de dependencias	`presentacion/contenedor.py`, `presentacion/api/dependencias.py`	Solo un lugar de la presentación conoce la infraestructura
Adapter	`presentacion/app.py` y `presentacion/api/`	Dos interfaces sobre los mismos servicios, sin duplicar reglas

Qué Cambió Frente a lo Planeado

La Tabla 7 compara el plan de la Semana 13 con lo que se hizo y explica cada diferencia.

Tabla 7

Diferencias entre lo planeado y lo realizado

Planeado	Lo que se hizo	Por qué
Una sola presentación de escritorio	API REST con FastAPI sobre los mismos servicios	La rúbrica de la Semana 15 pide endpoints REST; no hubo que cambiar las reglas de negocio para agregarla
E1: máximo 4 archivos	5 archivos de producción, uno por capa	Los casos de uso del servicio reciben los datos como parámetros explícitos, así que el servicio también cambió
Solo la migración 001	Migraciones 001 a 004	002 agrega la EPS (E1), 003 retira los objetos reemplazados y 004 repara `SP_AsignarJugadorPlantilla` en bases existentes

Planeado	Lo que se hizo	Por qué
Probar sobre todo con dobles en memoria	Pruebas de integración contra SQL Server real	Lo pidió la retroalimentación del Entregable 3. Encontraron dos defectos que los dobles no veían
Retirar 14 funciones sin uso y 2 duplicadas	41 definiciones retiradas: 39 funciones sin uso y las 2 duplicadas	La limpieza final de `legado/crud.py` abarcó más funciones que el diagnóstico inicial
Esquema preservado (Entregable 1)	La migración 001 agrega TUTOR.estado y la 002 agrega JUGADOR.eps	Son cambios aditivos que necesitan la baja de tutores y el escenario E1
Migración y protocolo sobre una copia de la base real	Sobre la copia de una base preparada por el equipo, con datos de prueba y ya migrada	No se contó con una copia de los datos reales sin migrar; la aplicación sobre una base nueva la cubren las pruebas de integración

Los dos defectos que encontraron las pruebas de integración ilustran por qué la retroalimentación era pertinente. El primero estaba en el script original `ClubFutbolDB.sql`: le faltaba un `GO` antes de `CREATE OR ALTER PROCEDURE SP_AsignarJugadorPlantilla`, así que las bases creadas con él conservaban la versión de tres parámetros y "Añadir titular" fallaba con el error 8144; se corrigió el script y la migración 004 repara las bases existentes. El segundo estaba en la unidad de trabajo: si la sesión se perdía, `rollback()` también fallaba y ese segundo error ocultaba el original; ahora se registra como advertencia y se propaga el error original.

Cómo Mejoran los Atributos de Calidad Priorizados

Los atributos priorizados en la Semana 8 fueron mantenibilidad, confiabilidad y seguridad (integridad del historial), más la eficiencia como escenario secundario (Bass et al., 2021; International Organization for Standardization [ISO], 2011; Salazar Bustamante et al., 2026b). Cada escenario se midió con un script del repositorio (`evaluacion/`) que ejecuta la misma operación sobre la versión original (esquema y código originales) y sobre la nueva, en el mismo servidor. La Tabla 8 resume los resultados.

Tabla 8

Resultados antes y después por escenario de calidad

Escenario	Medida	Antes	Después	¿Cumple?
E1 Mantenibilidad	Lecturas por posición en las pantallas del MVP	98	0	Sí
E1 Mantenibilidad	Archivos fuera del módulo Jugadores al agregar la EPS	—	0 (5 archivos, uno por capa)	Sí
E1 Mantenibilidad	Cobertura de pruebas del dominio y la aplicación (meta del Entregable 1: ≥ 70 %)	—	93–100 %	Sí
E2 Confiabilidad	Estados parciales en 10 fallos inyectados al cambiar el acudiente principal	10 de 10	0 de 10	Sí
E3 Seguridad	Filas de historial borradas al dar de baja a un profesor y a un jugador	57 (50 + 7)	0	Sí
E3 Seguridad	Criterios del protocolo de preservación del historial	—	6 de 6	Sí, con datos de prueba
E4 Eficiencia	Filas transferidas al abrir la edición, con 2000 registros	2000	1	Sí

Nota. Fuente: `docs/evaluacion/` del repositorio (`e1-mantenibilidad.md`, `resultados-mediciones.md`, `protocolo-historial.md`).

Mantenibilidad (E1)

El estímulo fue agregar la entidad de salud (EPS) al jugador: capturarla, editarla y verla en el listado. El cambio tocó cinco archivos de producción, uno por capa (migración 002, entidad, repositorio, servicio y pantalla), y ninguno fuera del módulo Jugadores. Antes, ese mismo cambio era arriesgado porque las pantallas del MVP tenían 98 lecturas por posición, cada una capaz de romperse en silencio al cambiar las columnas de una consulta; después hay cero. El tiempo del cambio no se registró, así que el criterio de menos de cuatro horas queda sin medir.

Confiabilidad (E2)

Se inyectó la pérdida de la conexión justo antes del segundo `UPDATE` de "cambiar acudiente principal", 10 veces con cada versión, contra SQL Server real. Con la versión original los 10 fallos dejaron al jugador sin acudiente principal; con la unidad de trabajo, ninguno. Las

pruebas de integración repiten el escenario con un error real del motor (violación del índice único) y con la sesión terminada mediante `KILL`, y en todos los casos el jugador conserva su acudiente principal y el fallo queda en la bitácora.

Seguridad: Integridad del Historial (E3)

Se dio de baja a un profesor con 12 sesiones (36 asistencias) y 2 plantillas, y a un jugador con pagos, morosidad, asistencias y su acudiente. La Tabla 9 muestra las filas borradas por cada versión. Además, se ejecutó el protocolo de preservación del historial del Entregable 1 (§3.5), que cumplió los seis criterios: preparación, conservación, exclusión de los desplegables, consulta histórica, reversibilidad e integridad referencial, sin filas huérfanas en 9 llaves foráneas. El protocolo se ejecutó sobre una base preparada por el equipo con datos de prueba, no sobre los datos reales del club.

Tabla 9

Filas de historial borradas al dar de baja

Tabla	Profesor: antes	Profesor: después	Jugador: antes	Jugador: después
SESION	12	0	0	0
SESION_JUGADOR	36	0	2	0
PLANTILLA_PROFESOR	2	0	0	0
PAGO	0	0	3	0
MOROSIDAD	0	0	1	0
JUGADOR_TUTOR	0	0	1	0
Total	**50**	**0**	**7**	**0**

Nota. Antes, el registro del profesor desaparecía; después sigue como inactivo, deja de aparecer en los desplegables y sus sesiones conservan su nombre.

Eficiencia (E4)

Se midieron la búsqueda por cédula y la apertura del formulario de edición con 50, 500 y 2000 registros. Antes, ambas operaciones transferían la tabla completa; después transfieren solo

la fila buscada, con una reducción de 98 % con 50 registros y cercana al 100 % con 2000 (Tabla 10). Los tiempos dependen del equipo; la cifra comparable es la de filas transferidas, que antes crecía con la tabla y ahora es constante.

Tabla 10

Filas transferidas y tiempo al abrir la edición de un jugador

Registros	Filas antes	Filas después	Reducción	ms antes	ms después
50	50	1	98,0 %	7,42	6,39
500	500	1	99,8 %	10,18	6,32
2000	2000	1	≈ 100 %	21,44	6,46

Nota. Mediana de 5 ejecuciones sobre el mismo servidor de SQL Server para ambas versiones. Tutores y profesores tienen el mismo comportamiento.

Comportamiento Conservado

Para confirmar que la usuaria no percibe cambios, las 18 operaciones del MVP tienen pruebas de caracterización: se conservan 11 comportamientos y se identifican 7 cambios deliberados, como la baja lógica en lugar del borrado. En total, `python -m pytest --integracion` ejecuta 166 pruebas (119 unitarias y 47 contra SQL Server), y la integración continua las repite en cada cambio.

Código Refactorizado y Repositorio

El código está en el repositorio de GitHub del proyecto (Salazar Bustamante et al., 2026d). El README contiene la descripción, los requisitos y dependencias (Python 3.10 o superior, SQL Server y el controlador ODBC 17 o 18), los pasos de instalación y ejecución, la tabla de endpoints con ejemplos en JSON, el enlace a la documentación de arquitectura (`docs/arquitectura.md`) y el acceso a Swagger. La Tabla 11 resume cómo se ejecuta cada parte.

Tabla 11

Ejecución del proyecto

Qué	Comando	Resultado
Dependencias	`pip install -r requirements.txt`	CustomTkinter, pyodbc, FastAPI, uvicorn, pytest y demás
Base de datos	`ClubFutbolDB.sql` y migraciones 001 a 004 en SSMS	Esquema del club actualizado
Aplicación de escritorio	`python run.py` o `ClubFutbol.exe`	Ventana de PaladinesFC
API REST	`python run_api.py`	API en `http://127.0.0.1:8000`, Swagger en `/docs`
Pruebas	`python -m pytest --integracion`	166 pruebas, 47 de ellas contra SQL Server
Mediciones	`python evaluacion/medir_escenarios.py`	`docs/evaluacion/resultados-mediciones.md`
Empaquetado	`pyinstaller ClubFutbol.spec`	`dist/ClubFutbol.exe`

El trabajo se organizó con issues en el tablero del proyecto, una rama por tarea y pull requests; a partir del #46, la integración continua ejecutó las pruebas unitarias y de integración en cada uno antes de fusionarlo. Para la entrega final se fusionaron los pull requests #41 a #52. Todos los integrantes tienen commits en el repositorio (Tabla 12).

Tabla 12

Aporte de cada integrante en la entrega final

Integrante	Aporte principal
Giany Andrés Salazar Bustamante	Pruebas de integración contra SQL Server (#12); API REST con FastAPI (#37); limpieza del legado, migraciones 003 y 004 y empaquetado (#19, #20); evidencias de funcionamiento (#40)
Mariana Aguilar Alcaraz	Escenario E1, campo EPS en un archivo por capa (#17); pruebas de caracterización de las 18 operaciones (#14); README final con la tabla de endpoints (#38)
Carlos Daniel Palacios Murillo	Integración continua con GitHub Actions y SQL Server 2022 (#13); mediciones antes/después de E2, E3 y E4 (#18); diagramas C4 versión 2, despliegue y secuencia (#21)
Juan Camilo Moreno Pérez	Migraciones sobre una copia de la base y protocolo de preservación del historial (#15, #16); sección de refactorización realizada y patrones aplicados (#39)

Evidencia de Funcionamiento

El repositorio incluye 22 capturas en `docs/evidencias/`, con su descripción y un guion para el video corto del flujo principal. Las figuras siguientes muestran las principales: la aplicación de escritorio con el MVP, la API en Swagger y las pruebas en la integración continua.

Figura 8

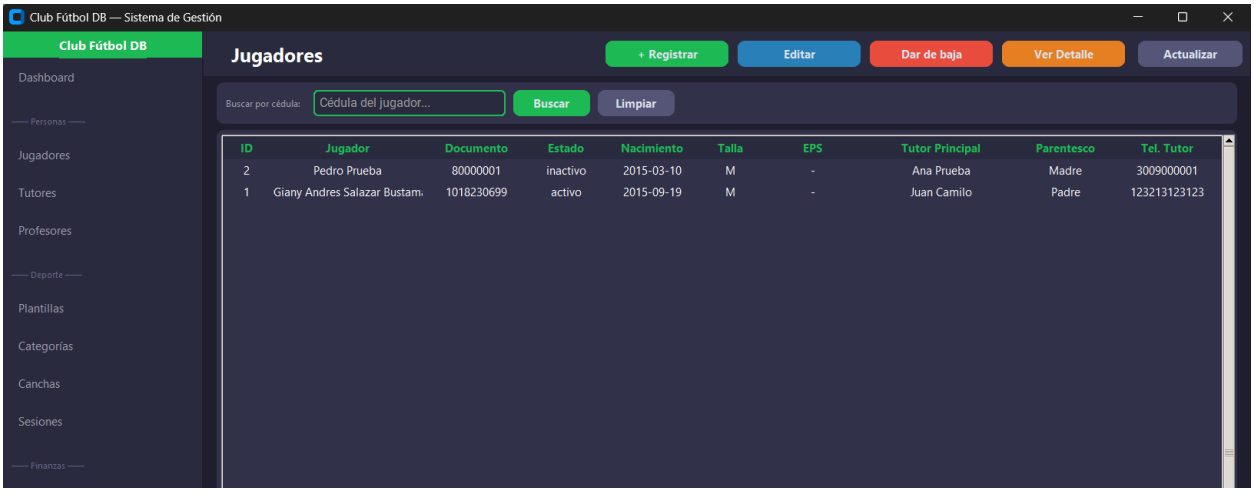
Listado de jugadores con la columna EPS del escenario E1



Nota. Captura de la aplicación de escritorio. El jugador dado de baja aparece como inactivo: no se borró (ADR-0003).

Figura 9

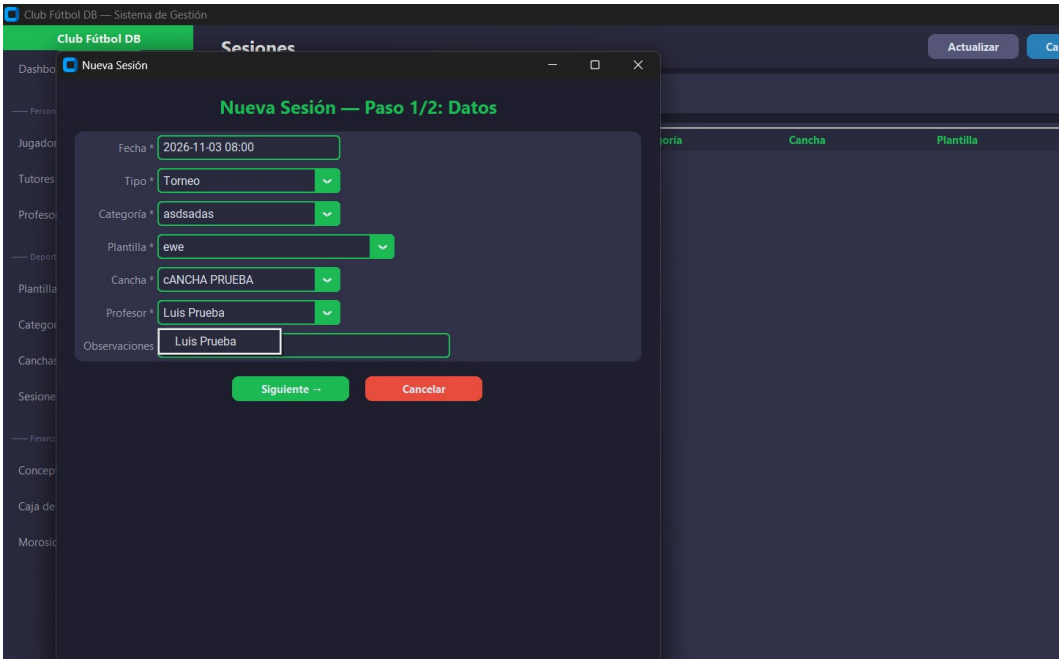
Aplicación empaquetada ClubFutbol.exe conectada a la base del club



Nota. Generada con `ClubFutbol.spec`; lee `config.ini` junto al ejecutable.

Figura 10

Nueva sesión: el desplegable solo ofrece profesores activos



Nota. Módulo legado de Sesiones después de la limpieza; muestra la exclusión operativa del escenario E3.

Figura 11

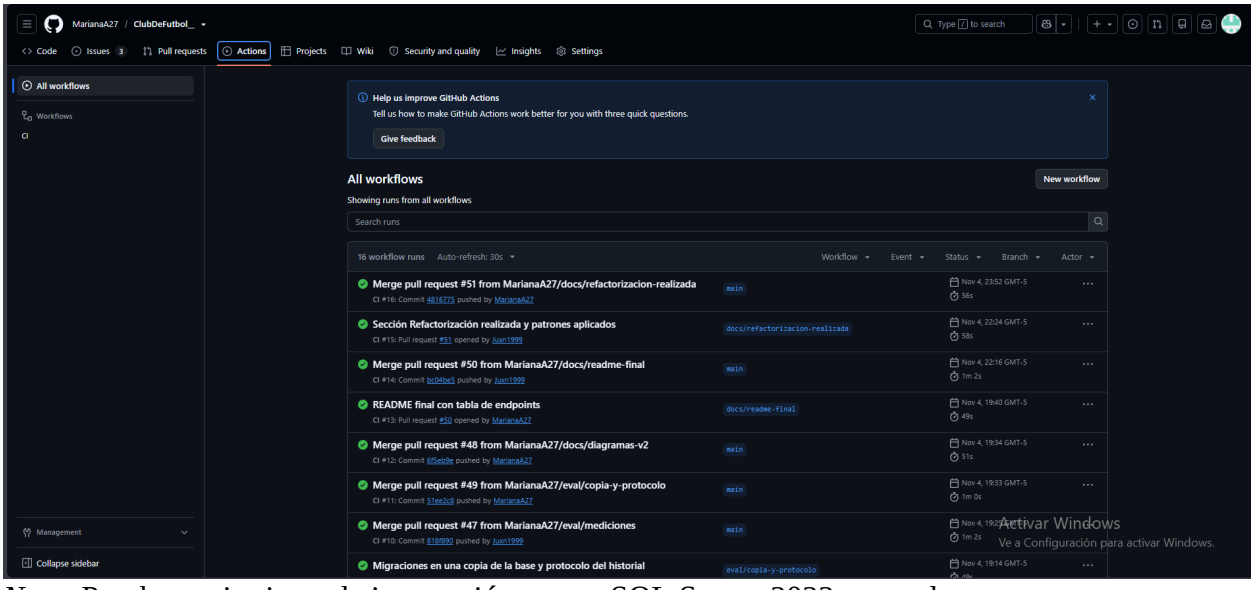
Documentación de la API REST en Swagger



Nota. `python run_api.py` y `http://127.0.0.1:8000/docs`. Cada endpoint se puede probar con Try it out.

Figura 12

Integración continua en GitHub Actions



Nota. Pruebas unitarias y de integración contra SQL Server 2022 en verde.

Conclusiones

La re-arquitectura resolvió los cinco problemas de la Semana 3 sin cambiar la experiencia de la usuaria. Separar el sistema en capas con inversión de dependencias dejó cada responsabilidad en un solo lugar: las reglas en el dominio, las transacciones en la unidad de trabajo, el SQL en los repositorios y las interfaces limitadas a llamar servicios. La mejor prueba de esa separación fue la API REST: se agregó una forma nueva de usar el sistema sin modificar el dominio, la aplicación ni la infraestructura.

Los atributos de calidad priorizados mejoraron de forma medible y en la misma operación: cero estados parciales frente a 10 de 10 en confiabilidad, cero filas de historial perdidas frente a 57 en seguridad, una fila transferida frente a la tabla completa en eficiencia y cero lecturas por posición frente a 98 en mantenibilidad. Validar contra SQL Server real, como pidió la retroalimentación, no solo confirmó esos resultados: encontró dos defectos que las pruebas con dobles no veían.

Quedan dos límites declarados: el tiempo del cambio de E1 no se registró y el protocolo de historial se ejecutó con datos de prueba. El siguiente paso natural es migrar los módulos restantes (Plantillas, Sesiones, Pagos y Morosidad) con el mismo patrón, que ya está probado en el MVP.

Referencias

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Brown, S. (s.f.). *The C4 model for visualising software architecture*. Recuperado el 5 de noviembre de 2026, de https://c4model.com

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

International Organization for Standardization. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011).

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Salazar Bustamante, G. A., Aguilar Alcaraz, M., Palacios Murillo, C. D., & Moreno Pérez, J. C. (2026a). *Selección de módulo MVP para re-arquitectura* [Trabajo de curso no publicado]. Facultad de Ingeniería, Universidad Católica Luis Amigó.

Salazar Bustamante, G. A., Aguilar Alcaraz, M., Palacios Murillo, C. D., & Moreno Pérez, J. C. (2026b). *Definición de atributos de calidad y escenarios arquitectónicos* [Trabajo de curso no publicado]. Facultad de Ingeniería, Universidad Católica Luis Amigó.

Salazar Bustamante, G. A., Aguilar Alcaraz, M., Palacios Murillo, C. D., & Moreno Pérez, J. C. (2026c). *Vistas de arquitectura, componentes y decisiones de refactorización* [Trabajo de curso no publicado]. Facultad de Ingeniería, Universidad Católica Luis Amigó.

Salazar Bustamante, G. A., Aguilar Alcaraz, M., Palacios Murillo, C. D., & Moreno Pérez, J. C. (2026d). *ClubDeFutbol__: Re-arquitectura de PaladinesFC* [Repositorio de software]. GitHub. https://github.com/MarianaA27/ClubDeFutbol__